



Reservations Module v1 – Runbook & Technical Documentation

1. Purpose & Scope

The **Reservations Module v1** is responsible for managing **time-bound room occupancy** within a hotel (tenant). It acts as the **single source of truth for booking timelines**, while synchronizing room physical state.

This module is **tenant-isolated**, **RBAC-controlled**, and **transaction-safe**.

2. Domain Model Overview

Tenant

- Represents a **Hotel**
- Owns Rooms, Users, and Reservations

Reservation (Aggregate Root)

A reservation represents a **claim on a room for a date range**.

Key characteristics: - Belongs to exactly **one tenant** - References exactly **one room** - Created by a **staff or admin user** - Lifecycle-driven (state machine)

3. Reservation Entity

Core Fields

Field	Description
id	UUID primary key
tenant	Tenant (Hotel)
room	Reserved room
createdBy	User who created reservation
guestName	Guest full name
guestEmail	Guest email
checkInDate	Inclusive date
checkOutDate	Exclusive date
status	Reservation lifecycle status

Field	Description
createdAt	Audit timestamp
updatedAt	Audit timestamp

Database Rules

- `checkInDate < checkOutDate` (DB CHECK constraint)
- Indexed by `(tenant, room, checkInDate, checkOutDate)`

4. Reservation Lifecycle (State Machine)

Allowed States

- PENDING
- CONFIRMED
- CHECKED_IN
- CHECKED_OUT
- CANCELLED

Allowed Transitions

From	To
PENDING	CONFIRMED, CANCELLED
CONFIRMED	CHECKED_IN, CANCELLED
CHECKED_IN	CHECKED_OUT
CHECKED_OUT	— (terminal)
CANCELLED	— (terminal)

Invalid transitions result in **400 Bad Request**.

5. Room Synchronization Rules

Room state is **derived**, not authoritative.

Reservation Status	Room Status
CHECKED_IN	OCCUPIED
CHECKED_OUT	AVAILABLE
Others	No change

Room updates are executed **inside a database transaction** together with reservation status changes.

6. Overlap & Availability Rules

Overlap Definition

Two reservations overlap if:

```
new.checkIn < existing.checkOut  
AND  
new.checkOut > existing.checkIn
```

Enforcement

- Only reservations with status:
- PENDING
- CONFIRMED
- CHECKED_IN are considered active
- Overlapping active reservations are **blocked**

Result: - 409 Conflict - Room already has an overlapping reservation

7. RBAC Matrix

Action	ADMIN	STAFF
Create reservation	✓	✓
View reservations	✓	✓
Change status	✓	✗
Check-in	✓	✗
Check-out	✓	✗

RBAC is enforced via @Roles + RolesGuard .

8. API Endpoints

Create Reservation

POST /reservations

- Roles: ADMIN, STAFF
- Status: 201 Created

List Reservations

GET /reservations

- Roles: ADMIN, STAFF
- Returns tenant-scoped list

Update Reservation Status

PATCH /reservations/:id/status

- Roles: ADMIN only
- Enforces lifecycle rules

9. Testing Runbook (curl + jq)

9.1 Login & Token

```
export ADMIN_TOKEN=$(curl -s -X POST http://localhost:3000/auth/login
-H "Content-Type: application/json"
-d '{"email":"admin@sunu.com","password":"password"}' | jq -r .accessToken)
```

9.2 Create Reservation

```
curl -s -X POST http://localhost:3000/reservations
-H "Authorization: Bearer $ADMIN_TOKEN"
-H "Content-Type: application/json"
-d '{
  "roomId": "<ROOM_ID>",
  "guestName": "John Doe",
  "guestEmail": "john@example.com",
  "checkInDate": "2026-03-10",
  "checkOutDate": "2026-03-12"
}' | jq
```

Expected Result

```
{
  "status": "PENDING"
}
```

9.3 Prevent Double Booking

```
curl -s -X POST http://localhost:3000/reservations
-H "Authorization: Bearer $ADMIN_TOKEN"
-H "Content-Type: application/json"
-d '{
  "roomId": "<ROOM_ID>",
  "guestName": "Overlap Guest",
  "guestEmail": "overlap@example.com",
  "checkInDate": "2026-03-11",
  "checkOutDate": "2026-03-13"
}' | jq
```

Expected Result

```
{
  "statusCode": 409,
  "message": "Room already has an overlapping reservation"
}
```

9.4 Lifecycle Transitions

PENDING → CONFIRMED

```
curl -s -X PATCH http://localhost:3000/reservations/<RES_ID>/status
-H "Authorization: Bearer $ADMIN_TOKEN"
-H "Content-Type: application/json"
-d '{"status":"CONFIRMED"}' | jq
```

CONFIRMED → CHECKED_IN

```
curl -s -X PATCH http://localhost:3000/reservations/<RES_ID>/status
-H "Authorization: Bearer $ADMIN_TOKEN"
-H "Content-Type: application/json"
-d '{"status":"CHECKED_IN"}' | jq
```

Expected

```
"room": { "status": "OCCUPIED" }
```

CHECKED_IN → CHECKED_OUT

```
curl -s -X PATCH http://localhost:3000/reservations/<RES_ID>/status
-H "Authorization: Bearer $ADMIN_TOKEN"
-H "Content-Type: application/json"
-d '{"status":"CHECKED_OUT"}' | jq
```

Expected

```
"room": { "status": "AVAILABLE" }
```

10. Definition of Done

The Reservations Module v1 is considered **DONE and LOCKED** when:

- All lifecycle transitions are validated
 - Overlapping reservations are blocked
 - Room status auto-sync works
 - RBAC is enforced
 - curl tests pass consistently
 - Documentation and runbook are complete
-

11. Current Status

✓ Reservations Module v1 is **production-stable** and frozen.

No refactor required before proceeding to next modules (Pricing, Housekeeping, Billing).